

Ella Potter

Professor Beichel

ECE:3360 Embedded Systems

Post-Lab Report 3

1. Introduction

The main goal for this lab was to extend our circuit from Lab 2, using the eight bit shift register, the push button and 7 segment display, by adding in a rotary pulse generator to now take care of incrementing/decrementing the hexadecimal display. The display should initially turn on and display “-“ and then increment from 0 to F when the user turns the RPG clockwise and decrement from F to 0 when the user turns the RPG counterclockwise. The user will turn the RPG to their desired value and then press the push button to enter that value into the code. If the 5 digit code entered is correct the yellow LED should be activated for four seconds, otherwise the display will show “_” and allow the user to re-enter the code after seven seconds.

2. Schematic

Figure 1 shows the circuit set-up for this lab using the same circuit as last time for the push-button, eight bit shift register, and seven segment display while adding in the rotary pulse generator as an input to the microcontroller. PB0, PB1, and PB2 are all outputs to the eight bit shift register to send the desired display value. The push button is again implemented by the circuit given in the ES25_Lab02 document and is connected to PB3 as an input. The RPG is set up for pin A to send input to PD2 and pin B to send input to PD3. It is also properly set up with a hardware debounce using two pull-up resistors and two low pass filters to ensure that the signal is being properly outputted to the microcontroller.

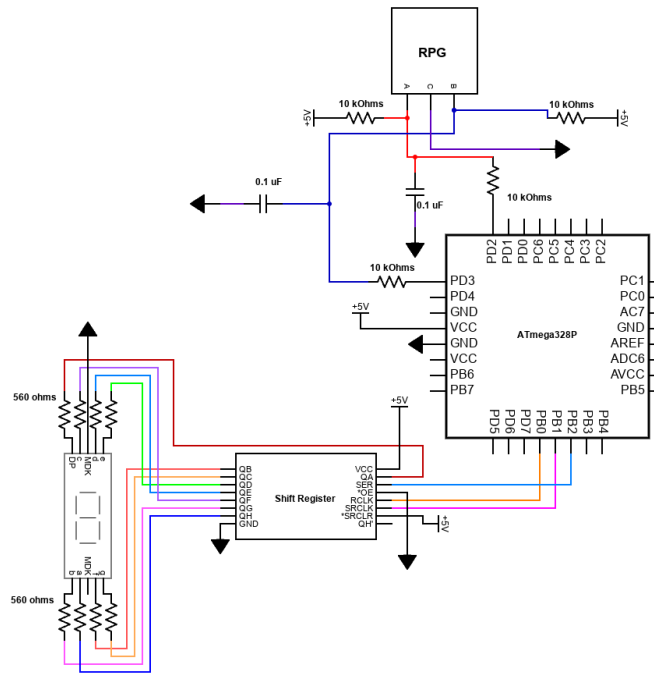


Figure 1: Lab 3 Circuit

3. Discussion

This lab mainly focused on the software component rather than the hardware as most of the circuit was already set up for last time. But, I will still briefly discuss what each component is doing before moving onto the software,

3.1 Hardware

The circuit contained 4 components that were already used and set up in the last lab.

- **ATmega328P:** used to send the correct display code to the shift register as well as take in input from the push button and RPG to know both when the user is incrementing/decrementing the display or is trying to enter a digit into the code
- **SNx4HX595:** used to send 8-bit data to the 7-segment display letting it know what segments to light up
 - **SER:** carries the data one bit at a time from the microcontroller to the shift-register
 - **SRCLK:** controls when a data bit is moved into SER (rising edge)
 - **RCLK:** latches the data in the register until all 8 bits loaded then will send the data out to the 7 segment display

- **5161AS:** used to display each hexadecimal digit, each display segment will receive a 0 or 1 from the shift register letting it know whether or not to light up
- **Push-Button:** user can press the button to pick the correct code digits
 - **Button Press < 1 sec:** enter digit into code
 - **Button Press > 2 sec:** reset the counter to “-“
- **Rotary Pulse Generator:** used to send output to the microcontroller when the user is turning the knob. Sends two bits, one from pin A and one from pin B
 - **Clockwise:** when the RPG is being turned clockwise it will output 00-01-11-10-00 for each detent
 - **Counter-Clockwise:** when the RPG is being turned counter clockwise it will output 00-10-11-01-00

3.2 Software

After adding the RPG into the circuit, I then needed to start writing the code to be able to take in input from the RPG, determine which way it was being turned, and correctly update the display value based on this information. It is important to note when looking at the RPG output that the values will be opposite of what was stated in hardware section, so when the RPG is outputting 0 it will actually be at logic high outputting 1. After determining what way the RPG was turned I would either need to increase or decrease my index for my lookup table to be able to retrieve the correct output. After completing that step I would need to implement the new feature we learned about on the AT mega, timers, which would allow me to keep track of how long the button was pressed down for. To be able to implement the timer I would need to calculate the correct prescaler and count value to make sure it delayed for the correct amount of time. The final step would be to make sure that we can keep track of the digits being entered and after 5 digits are entered compare the entered digits with the correct code.

*Note: I would like to add that I had some partner difficulties during this lab and was not able to fully implement all the given requirements, but I will still go over how I would implement them if I had more time. (I talked to professor about this as well)

Function	Description
Main Loop	In the main loop, the code will continuously check PIND which is the output from the RPG and if the input turns 0 it will begin to check the rotation direction. This happens when the RPG is in the middle of a turn cycle.
Check Rotation	If the code detects that the RPG has outputted 0 it will then receive the next input from the RPG and mask it to make sure we only receive the two values we want. If this new value from the RPG shows that only A is high the code will branch to the

	clockwise loop, otherwise if it realizes only B is high it will branch to counterclockwise.
CW/CCW	In the clockwise and counterclockwise section the code first will load the correct mode into the mode register so that we know whether or not to increment or decrement. It will then read the final state from the RPG and ensure that both values are high, meaning the full rotation has been made. If this is true and both values are high it will finally update the display.
Update Display Increment/Decrement	When update display is called it will check what mode the register is in (incrementing or decrementing) and branch to the correct loop based on that value. In the increment and decrement loops the code will either increase currentDig (our index for the lookup table) or decrease currentDig and then call outputDisplay
Output Display	Output Display will first check that the current digit is not already at F and trying to index over what we are allowed. It will then load the look up table into the Z register and use currentDig to index to the correct 7 segment display pattern. It will then load this data into register 16 and call display to send the data to the 7 segment display
Delay (Timer)	For the delay loop I used the code given from the ES25_06_Timers.pdf and calculated the correct values to have a 500 us delay that would then be able to be used to keep track of how long the button was pressed down using similar logic from my Lab 2 code. (Timer Calculations Below)

Timer Calculation:

We know we have a clock frequency of 16 MHz and we want to find a prescalar and count value that will get us a 500 us delay.

Prescalar	Frequency
1	16 MHz
8	$16\text{MHz} / 8 = 2\text{MHz}$
64	$16\text{ MHz} / 64 = 250\text{ kHz}$
256	$16\text{ MHz} / 256 = 62.5\text{ kHz}$

If we choose the prescalar of 256, we can then calculate our N value as $62.5\text{ kHz} * 500\text{ us} = 31.25$. From there we can find the count value by taking $256 - 31.25 = 224.75$.

Therefore, if we use a prescalar of 256 and a count value of 225 we will get a 500 us delay from our timer. Although not implemented, I would be able to then use this as a base delay that can be looped through multiple times to create longer delay lengths. Then with my code from last lab I call delay while the button is held down and keep track of how many loops I made. If this number was under 1 second I would load the value currently in register 16 (the display register) into a register keeping track of my entered codes. If this was also the fifth code to be entered in, I would compare all digits of this

code with the correct code and then correctly handle if it was the correct or incorrect code. If the number was over 2 seconds I would simply call the reset function from my last lab and add in a part to display “-“ instead of 0 at the beginning.

4. Conclusion

From this lab, I have gained more experience on using multiple inputs at the same time with the microcontroller, and being able to correctly differentiate between these using a mask. I have also learned the importance of timers and how they can be easily manipulated to perform delays of different lengths when needed. Another important thing I learned was not to wait until the last minute to discuss concerns with my partner as I truly believe I would have been able to implement this fully by myself if I had given myself more time.

5. Appendix A: Source Code

```

6. .include "m328Pdef.inc"
7. .org 0x00
8.     rjmp start
9.
10. .org 0x100
11. inc_table:
12.     .db 0xFA, 0x60, 0xDC, 0xF4, 0x66, 0xB6, 0xBE, 0xE0
13.     .db 0xFE, 0xE6, 0xEE, 0x3E, 0x9A, 0x7C, 0x9E, 0x8E ; Hex values to show 0-F on
        display
14.
15. .def displayVal = r16
16. .def currentDig = r18
17. .def currentState = r19
18. .def mode = r20
19. .def count = r22
20. .def tmp1 = r23
21. .def tmp2 = r24
22.
23. .equ AHigh = 0x4
24. .equ BHigh = 0x8
25. .equ BothHigh = 0xC
26. .equ mask = 0b00001100
27.
28.
29. start:
30.
31. .cseg
32.
33. ; === Initialization ===
34. sbi DDRB, 0 ; PB0 = SER output (to SN74HC595)
35. sbi DDRB, 1 ; PB1 = SRCLK output (to SN74HC595)
36. sbi DDRB, 2 ; PB2 = RCLK output (to SN74HC595)
37. cbi DDRD, 2 ; PD2 = RPG A input
38. cbi DDRD, 3 ; PD3 = RPG B input
39.
40. ldi count, 225
41. ldi tmp1, 0x04

```

```

42. out TCCR0B, tmp1 ;configure timers
43. ldi tmp1, 0x00
44. out TCCR0A, tmp1
45. rcall delay
46. ldi ZH, high(inc_table << 1) ; Load high byte of lookup table
47. ldi ZL, low(inc_table << 1) ; Load low byte of lookup table
48. ldi mode, 0x0
49. ldi currentDig, 0x00
50. ldi displayVal, 0x4
51. rcall display
52.
53. main_loop:
54.     in currentState, PIND
55.     andi currentState, mask
56.     cpi currentState, 0x00
57.     breq check_rotation
58.     rjmp main_loop
59.
60. check_rotation:
61.     rcall delay
62.     in currentState, PIND
63.     andi currentState, mask
64.     cpi currentState, AHigh
65.     breq cw
66.     cpi currentState, BHigh
67.     breq ccw
68.     rjmp check_rotation
69.
70. cw:
71.     ldi mode, 0x0
72.     rcall delay
73.     in currentState, PIND
74.     andi currentState, mask
75.     cpi currentState, BothHigh
76.     breq update_display
77.     rjmp cw
78.
79. ccw:
80.     ldi mode, 0x1
81.     rcall delay
82.     in currentState, PIND
83.     andi currentState, mask
84.     cpi currentState, BothHigh
85.     breq update_display
86.     rjmp ccw
87.
88. update_display:
89.     cpi mode, 0x0
90.     breq increment
91.     brne decrement
92.
93. increment:
94.     inc currentDig
95.     rjmp outputDisplay
96. decrement:
97.     dec currentDig
98.     rjmp outputDisplay
99.

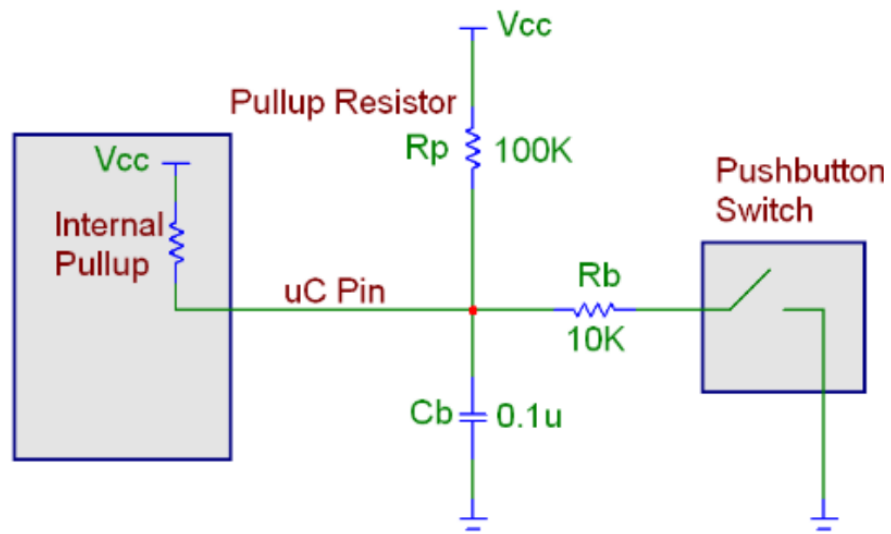
```

```

100. outputDisplay:
101.     cpi currentDig, 0x10
102.     brsh main_loop
103.     ldi ZH, high(inc_table << 1) ; Load high byte of lookup table
104.     ldi ZL, low(inc_table << 1)  ; Load low byte of lookup table
105.     add ZL, currentDig
106.     lpm displayVal, Z             ; Load segment pattern from lookup table
107.     rcall display                ; Call display function to update 7-segment
108.     rjmp main_loop
109.
110.
111. display:
112.     push R16
113.     push R17
114.     in R17, SREG
115.     push R17
116.     ldi R17, 8 ; loop --> test all 8 bits
117. loop:
118.     rol R16 ; rotate left through Carry
119.     BRCS set_ser_in_1 ; branch if Carry is set
120.     cbi PORTB, 0 ; SER = 0
121.     rjmp end
122. set_ser_in_1:
123.     sbi PORTB, 0 ; SER = 1
124. end:
125.     sbi PORTB, 1 ; Generate SRCLK pulse
126.     cbi PORTB, 1
127.     dec R17
128.     brne loop
129.     sbi PORTB, 2 ; Generate RCLK pulse
130.     cbi PORTB, 2
131.     pop R17
132.     out SREG, R17
133.     pop R17
134.     pop R16
135.     ret
136.
137. delay:
138.     in TMP1, TCCR0B
139.     ldi tmp2, 0x00
140.     out TCCR0B, tmp2
141.
142.     in tmp2, TIFR0
143.     sbr tmp2, 1<<TOV0
144.     out TIFR0, tmp2
145.
146.     out TCNT0, count
147.     out TCCR0B, tmp1
148.
149. wait:
150.     in tmp2, TIFR0
151.     sbrs tmp2, TOV0
152.     rjmp wait
153.     ret

```

154. Appendix B: References

*Push-Button Circuit*